

Sound Realty ML Model Serving

Technical Architecture & Implementation Deep Dive

ML Engineering Team

2026

Project Overview

- **Goal:** Deploy a KNN model for home price predictions
- **Input Data:** 18 property features + zipcode demographics
- **Model:** K-Nearest Neighbors ($k=5$) with RobustScaler normalization
- **Stack:** FastAPI, Python, scikit-learn, Docker
- **Architecture:** REST API with hot-reload model capability
- **Outcome:** Production-ready service with comprehensive testing & monitoring

Guidelines for developing the architecture and implementation

- **KISS:** Keep It Simple, Stupid - focus on core functionality. An MVP for a new client should prioritize reliability and maintainability over complex features.
- **Input Data:** make validation robust, but no need for complex feature engineering
- **Model:** Model improvement is a bonus, but focus on solid KNN implementation
- **Architecture:** keep it modular and easy to test locally

Deployed Architecture Overview

images/architecture_diagram.png

Data Pipeline & Feature Engineering

Data Sources:

- KC House Sales (21,613 records)
- Zipcode Demographics
- Feature-engineered variants

Feature Set: Primary (bedrooms, bathrooms, sqft_living, grade), Secondary (lat/long, year_built), Demographic (population, income, education)

Data Integration:

```
# Merge demographics by zipcode  
merged = sales.merge(demographics, how='left', on='zipcode')
```

Preprocessing: RobustScaler (handles outliers), 80-20 train-test split, Feature importance analysis

Model Architecture: KNN Regression

Why KNN?

- **Interpretability:** Predictions based on k nearest training samples
- **Non-parametric:** Captures complex non-linear relationships
- **No training phase:** Instantaneous retraining possible
- **Baseline:** okay performance on housing datasets

Model Pipeline:

RawFeatures $\xrightarrow{\text{RobustScaler}}$ NormalizedFeatures $\xrightarrow{\text{KNN (k=5)}}$ Price Prediction

Performance Metrics:

- Test RMSE: \$175K-\$185K (varies by feature set)
- Test MAE: \$105K-\$115K
- Test R^2 Score: 0.82-0.84
- MAPE: 22-24%

Model Versioning & A/B Testing

Multiple Model Variants:

- **Basic Model:** 10 numeric features only
- **Added Features Model:** 18 features + demographics
- **Future Variants:** Gradient Boosting, Neural Networks

Model Storage Structure:

```
model/  
    basic/  
        model.pkl (KNN pipeline)  
        model_features.json  
        metrics.json  
    added_features/  
        model.pkl  
        model_features.json  
    2026-03-05_05/ (timestamp-based)
```

Hot-Reload Capability: Switch models via environment variable without API restart

API Design: REST Endpoints

Endpoint #1: Health Check

```
GET /health      {status: 'healthy', models_loaded: true}
```

Endpoint #2: Full Prediction (18 fields)

```
POST /predict
{
  bedrooms, bathrooms, sqft_living, grade, ..., zipcode
}

{prediction: $850K, model_version: 'added_features'}
```

Endpoint #3: Minimal Prediction (8 fields)

```
POST /predict-minimal (uses basic model)
```

API Framework: FastAPI

- Async/await for concurrent request handling
- Pydantic validation with automatic OpenAPI docs
- Lifespan context manager for startup/shutdown

Monitoring & Observability

Model Call Logging:

- Every prediction logged with timestamp, input features, output
- Client metadata: IP, user-agent, endpoint
- Latency tracking for performance monitoring
- Structured JSON format for analysis

What We Track:

```
{  
  "timestamp": "2026-03-05T10:23:45Z",  
  "model_version": "added_features",  
  "input_features": {...},  
  "prediction": 850000,  
  "latency_ms": 12.5,  
  "client_ip": "192.168.1.1"  
}
```

Benefits:

Testing Strategy

Test Coverage (pytest):

- **API Tests:** Endpoint validation, edge cases, error handling
- **Model Tests:** Prediction outputs, model switching, feature loading
- **Integration Tests:** Full pipeline (data → prediction)
- **Logger Tests:** Call logging, JSON serialization

Key Test Scenarios:

- Invalid input handling (missing fields, type mismatches)
- Model hot-reload (switching between versions)
- Prediction consistency (deterministic output)
- Data processing (demographics merge, feature ordering)

CI/CD: pytest automatically runs on every commit

Containerization & Deployment

Docker Strategy:

```
FROM python:3.10-slim
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0"]
```

Docker Compose Orchestration:

- FastAPI service on port 8000
- Nginx reverse proxy on port 80
- Volume mounts for model persistence
- Environment-based model selection

Production Considerations:

- Stateless design for horizontal scaling
- Lazy loading to minimize startup time
- Graceful shutdown with lifespan context

Model Performance (Test Set):

Metric	Basic	Added Features	Unit
RMSE	180K	175K	\$
MAE	110K	105K	\$
R ² Score	0.82	0.84	-
MAPE	23%	22%	%

Bias Analysis:

- Underpricing occurs 18% of the time (bad for sellers)
- Weighted RMSE penalizes underprediction 2x
- Feature importance: sqft_living & grade dominate

Inference Latency: 8-15ms per prediction (GPU not required)

Engineering Decisions & Trade-offs

Decision 1: KNN over Gradient Boosting

- **Pro:** Hot-reload capability, interpretability, fast inference
- **Con:** Slower than GB for very large feature spaces; memory-dependent

Decision 2: RobustScaler over StandardScaler

- **Reasoning:** Housing prices have outliers; RobustScaler is median-based

Decision 3: Lazy Loading of Models

- **Pro:** Fast startup, multiple models in memory
- **Con:** First request slightly slower

Decision 4: Structured Logging

- JSON logs enable machine-readable analysis, drift detection

Production Monitoring Dashboard

Key Metrics to Track:

- **Throughput:** Requests/second
- **Latency:** P50, P95, P99 latencies
- **Model Drift:** Mean absolute error of recent predictions vs. baseline
- **Data Quality:** % of invalid inputs caught by validation
- **Prediction Distribution:** Are we making reasonable price estimates?

Alerting Thresholds:

- API latency $\geq$ 100ms
- Error rate $\geq$ 1%
- Prediction mean shifts $\geq$ 5% from baseline
- Memory usage $\geq$ 80%

Retraining Trigger: Model drift detected (weekly comparison) or RMSE degrades

Questions for Technical Discussion

Potential Discussion Topics:

- ① Why not use dimensionality reduction (PCA) given 18 features?
- ② How would you handle categorical features (e.g., zipcode as categorical)?
- ③ What's your strategy for handling missing demographics data?
- ④ How do you prevent data leakage in train-test split?
- ⑤ What about outlier detection before prediction?
- ⑥ How would you implement online learning / continual retraining?
- ⑦ Prediction intervals vs. point estimates?